

```

ocl_status = clGetDeviceIDs(ocl_platforms[0], CL_DEVICE_TYPE_
ALL, 0,
    NULL, &num_oclDevices);
ocl_devices = (cl_device_id*)malloc(num_oclDevices*sizeof(cl_
device_id));
/* looking for all opencl device types */
ocl_status = clGetDeviceIDs(ocl_platforms[0], CL_DEVICE_TYPE_
ALL,
    num_oclDevices, ocl_devices, NULL);
ocl_devices = (cl_device_id*)malloc(num_oclDevices*sizeof(cl_
device_id));
/* getting the device ids */
ocl_status = clGetDeviceIDs(ocl_platforms[0], CL_DEVICE_TYPE_
ALL,
    num_oclDevices, ocl_devices, NULL);

/* creating a context and associate all opencl devices to it
*/
ocl_context = clCreateContext(NULL, num_oclDevices, ocl_
devices, NULL,
    NULL, &ocl_status);
/* creating a command queue for enqueuing the kernel for
execution */

ocl_cmdQueue = clCreateCommandQueue(ocl_context, ocl_
devices[0], 0,
    &status);

/* creating buffer objects to hold the data on device for
computations */
cl_mem device_vect_a, device_vect_b, device_vect_c;

device_vect_a = clCreateBuffer(ocl_context, CL_MEM_READ_ONLY,
    sizeof(int)*vect_size, NULL, &ocl_status);
device_vect_a = clCreateBuffer(ocl_context, CL_MEM_READ_ONLY,
    sizeof(int)*vect_size, NULL, &ocl_status);

/* create and compile the program */
cl_program ocl_program = clCreateProgramWithSource(ocl_
context, 1,
    (const char**)&ocl_programSource, NULL, &ocl_status);

ocl_status = clBuildProgram(ocl_program, num_oclDevices,
ocl_devices, NULL,
    NULL, NULL);

/* creating the kernel "vectoraddition" */
cl_kernel ocl_dkernel = NULL;
ocl_dkernel = clCreateKernel(ocl_program, "vectoraddition",
    &ocl_status);
/* setting the kernel arguments */
ocl_status = clSetKernelArg(ocl_dkernel, 0, sizeof(cl_mem),
    &device_vect_a);
ocl_status = clSetKernelArg(ocl_dkernel, 1, sizeof(cl_mem),
    &device_vect_b);

ocl_status = clSetKernelArg(ocl_dkernel, 2, sizeof(cl_mem),
    &device_vect_c);
globalWorkSize[0] = elements;

/* enqueue the kernel for execution on device */
ocl_status = clEnqueueNDRangeKernel(ocl_cmdQueue, ocl_
dkernel, 1, NULL,
    globalWorkSize, NULL, 0, NULL, NULL);

/* Read the output vector from device */
clEnqueueReadBuffer(ocl_cmdQueue, device_vect_c, CL_TRUE, 0,
    sizeof(int)*vect_size, vect_c, 0, NULL, NULL);
/* results of the computation are stored in vect_c */

/* free all OpenCL resources */
clReleaseKernel(ocl_dkernel);
clReleaseProgram(ocl_program);
clReleaseCommandQueue(ocl_cmdQueue);
free(ocl_platforms); free(ocl_devices);

/* free the device memory used for computation */
clReleaseMemObject(device_vect_a);
clReleaseMemObject(device_vect_b);
clReleaseMemObject(device_vect_c);
clReleaseContext(ocl_context);

/* free the host resources */
free(vect_a); free(vect_b); free(vect_c); }

```

The above program can be compiled using the make file or using the below command:

```

g++ -L/usr/lib64 -I/usr/local/cuda/include -lOpenCL vector_
vector_addition.c -o vect-vect-addition

```

On successful compilation of the code, the single executable i.e vect-vect-addition is created. Execute this binary in shell.

```

[user@system01] ./vect-vect-addition 

```

References

- [1] For OpenCL specifications, reference and man pages, refer to <https://www.khronos.org/opencl/>
- [2] 'Heterogeneous Computing with OpenCL' by Benedict R. Gaster, Lee Howes, David Kaeli, Perhaad Mistry and Dana Schaa
- [3] Intel SDK for OpenCL: <https://software.intel.com/en-us/articles/using-the-intel-sdk-for-opencl-applications-xe-2013>
- [4] OpenCL resources @ NVIDIA: <https://developer.nvidia.com/opencl>

By: Shivaprasad Katta

The author is a software engineer at Dell India's R&D Centre. His interests are virtualisation, cloud computing and HPC clusters.